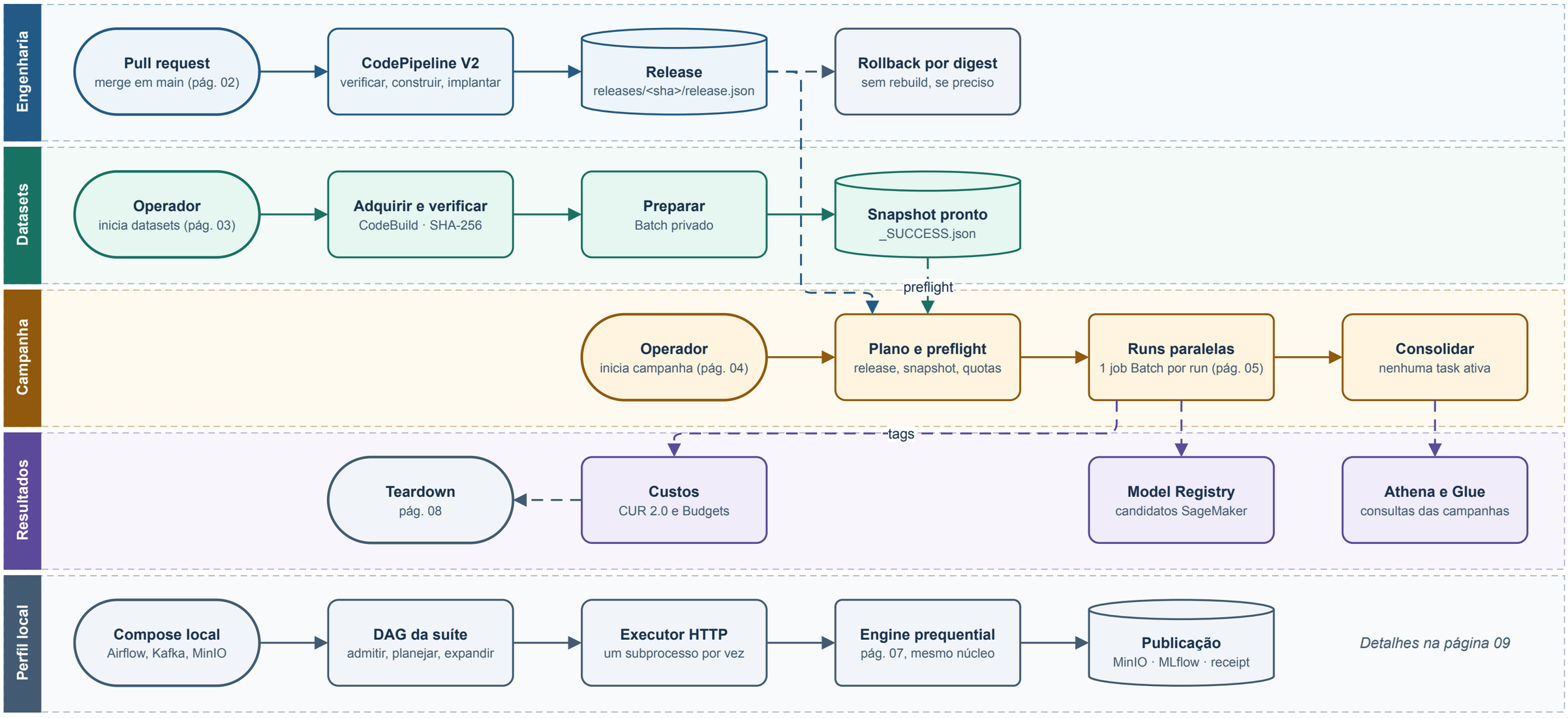
Visão geral do ciclo de vida

Da mudança de código ao resultado consultável: quatro fluxos independentes no perfil AWS e um perfil local.



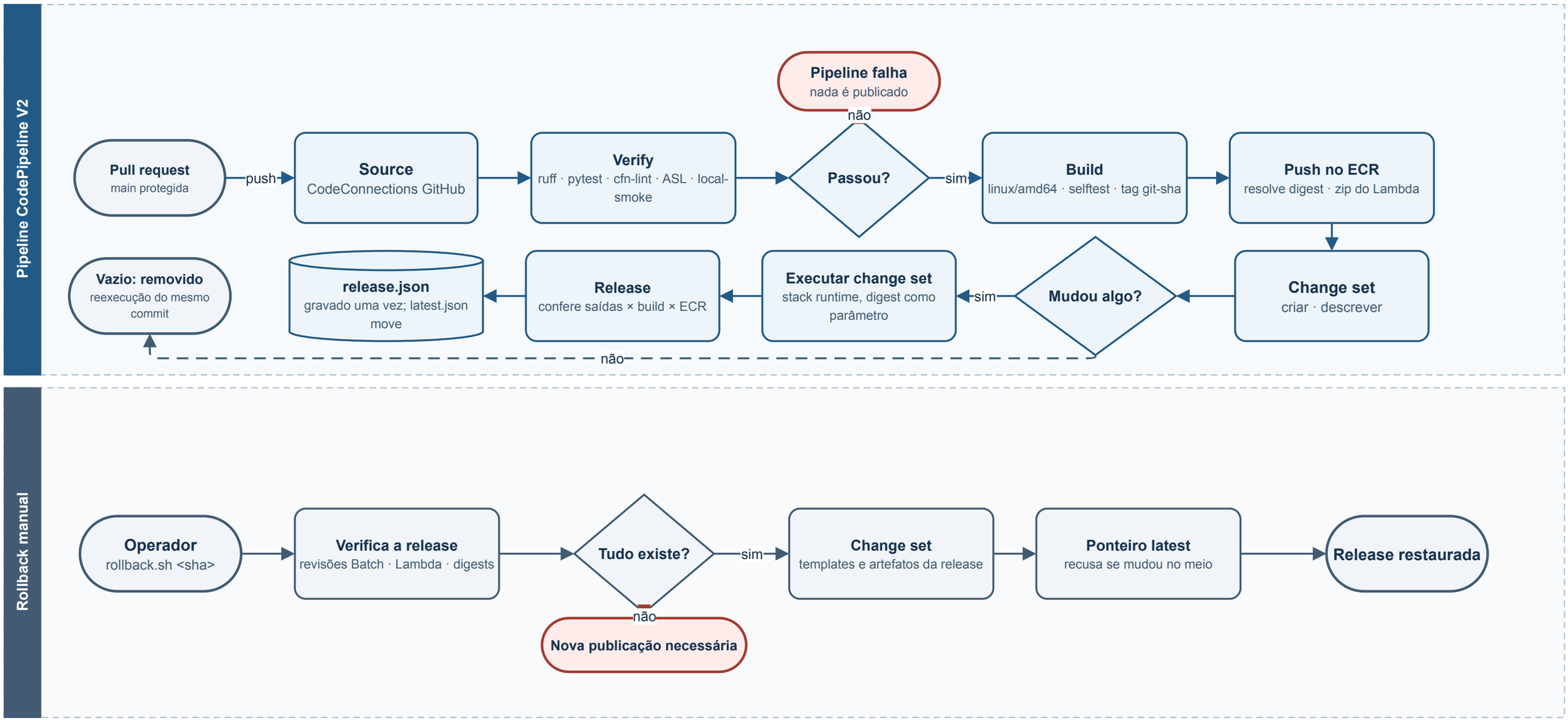
O merge só publica imagens e infraestrutura. Datasets e campanhas são sempre iniciados manualmente pelo operador.

Cada campanha resolve a release uma única vez, no plano, e conserva essa versão até o fim, mesmo que outro merge aconteça.

O perfil local (Compose, Airflow, Kafka, MinIO, MLflow) não depende da AWS e usa o mesmo núcleo de avaliação.

Do commit à release

Um único gatilho publica imagens imutáveis e atualiza a infraestrutura. O rollback reaproveita a release anterior.



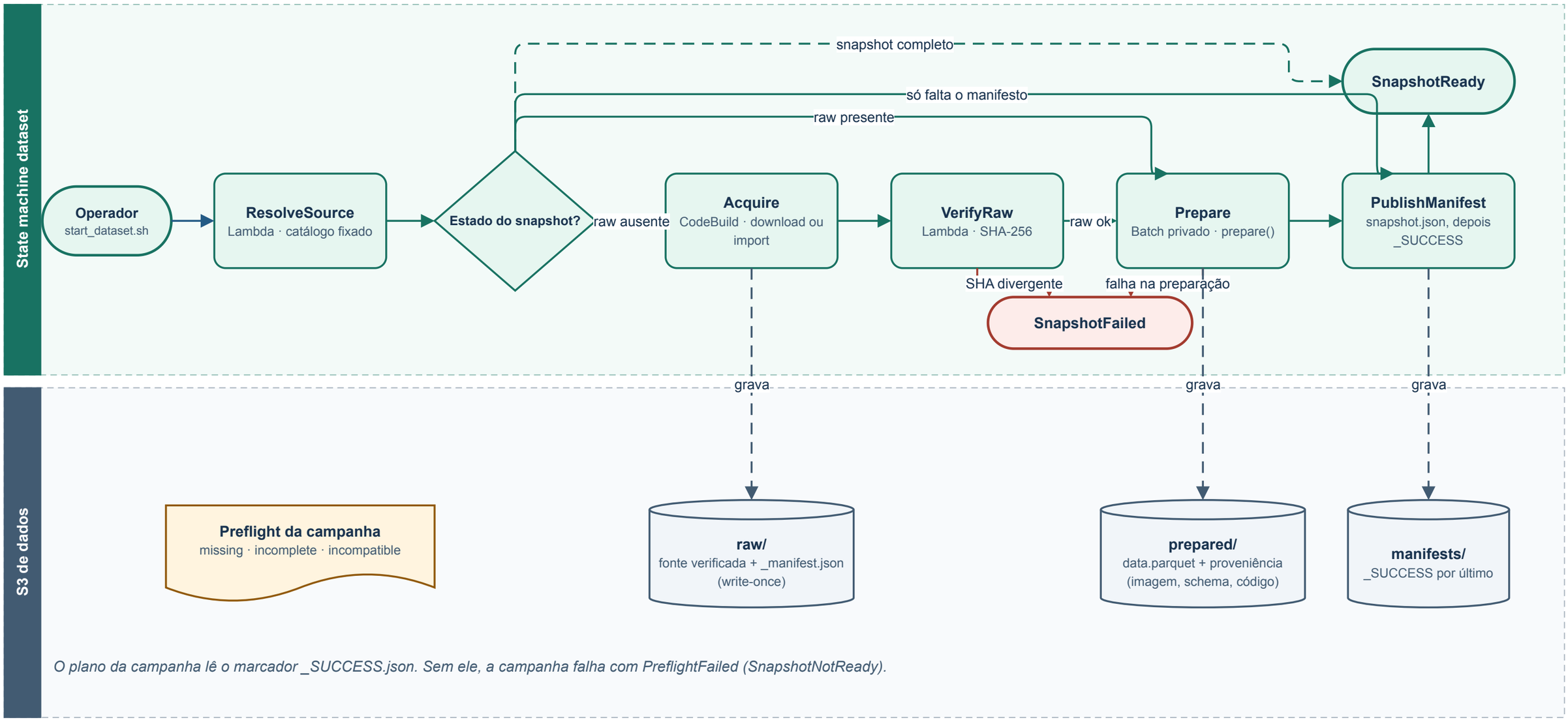
O gatilho é push filtrado para main. Não há webhook nem polling extra: uma revisão nunca dispara duas execuções.

ExecutionMode SUPERSEDED prioriza a execução mais recente e pula execuções intermediárias obsoletas.

Os buildspecs não contêm start-execution, acquire nem submit-job. Um teste garante que o merge não inicia campanhas.

Aquisição e preparação de datasets

A state machine de datasets reaproveita cada etapa já concluída e só marca o snapshot pronto depois do _SUCCESS.json.



O plano da campanha lê o marcador _SUCCESS.json. Sem ele, a campanha falha com PreflightFailed (SnapshotNotReady).

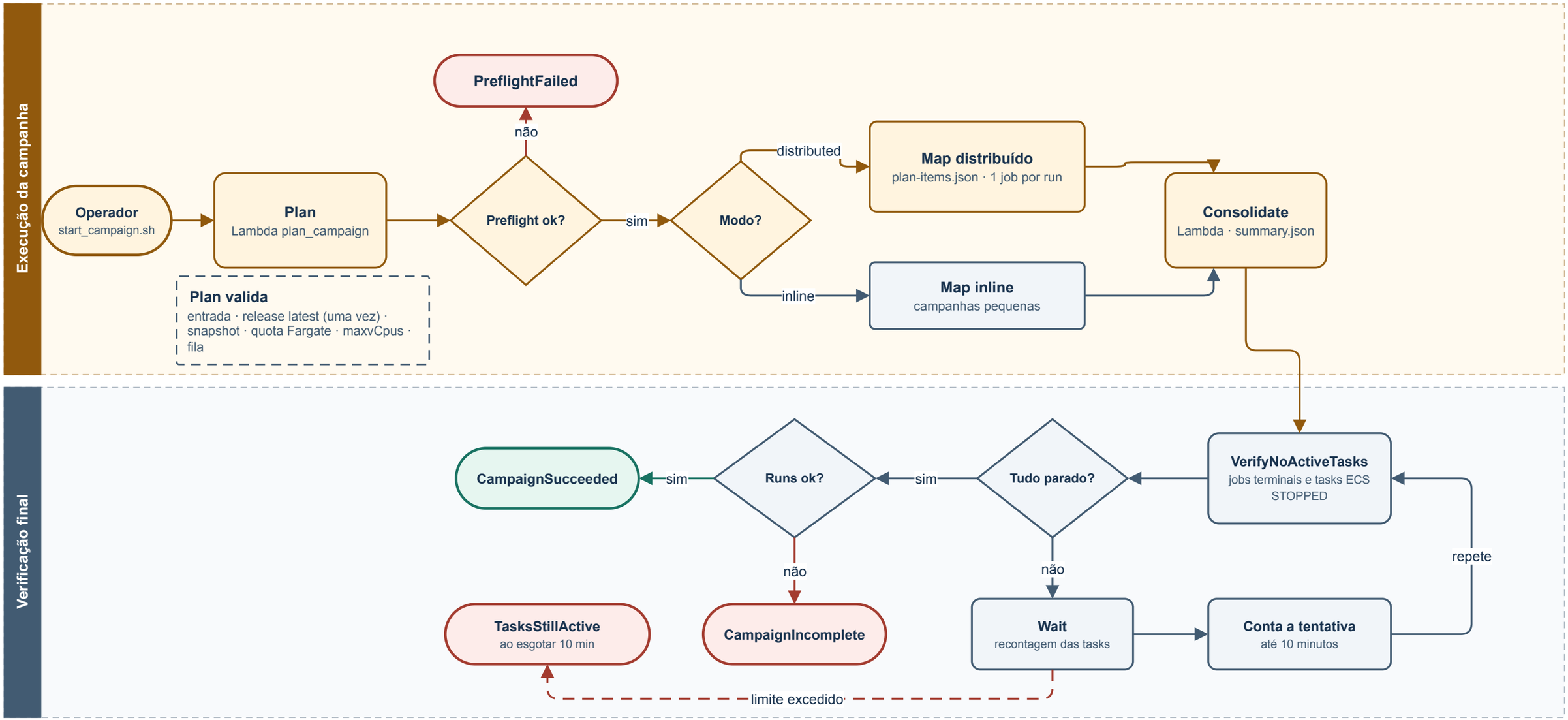
Modos: download (CodeBuild fora da VPC), import (copia de imports/ no S3, só lê esse prefixo) e prepare (usa raw já publicado).

Execuções com o mesmo nome são recusadas enquanto a anterior roda. Para retomar, repita a entrada com um sufixo.

Objetos válidos são reaproveitados e evidência divergente nunca é sobrescrita. O benchmark exige snapshot completo no preflight.

Campanha de benchmark na AWS

O plano valida tudo antes de gastar: release, snapshot, quota e fila. Depois as runs rodam em paralelo e o fim exige zero tasks ativas.



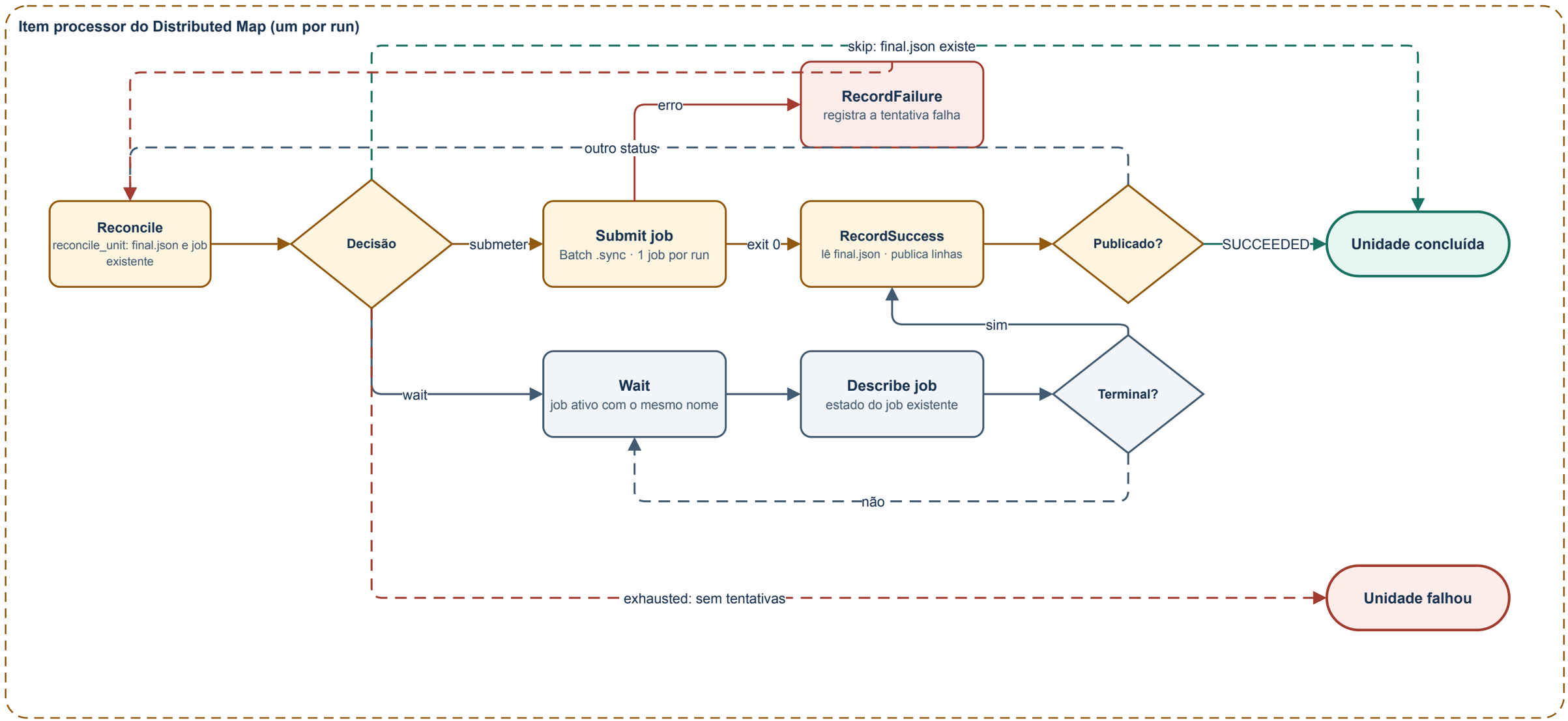
O nome da execução é o campaign_id. Reexecutar usa <campaign_id>--<sufixo> e reaproveita as runs que já têm manifesto final.

Cada unidade do Map é detalhada na página 05. ToleratedFailurePercentage é 100: quem decide o resultado da campanha é a consolidação.

Preflight falha com SnapshotNotReady se o snapshot estiver ausente, sem marcador ou divergente (página 03).

Uma unidade do Map: reconciliar, submeter, registrar

Antes de cada submissão a unidade consulta o que já existe. Isso evita reenvio quando a resposta de uma submissão se perde.



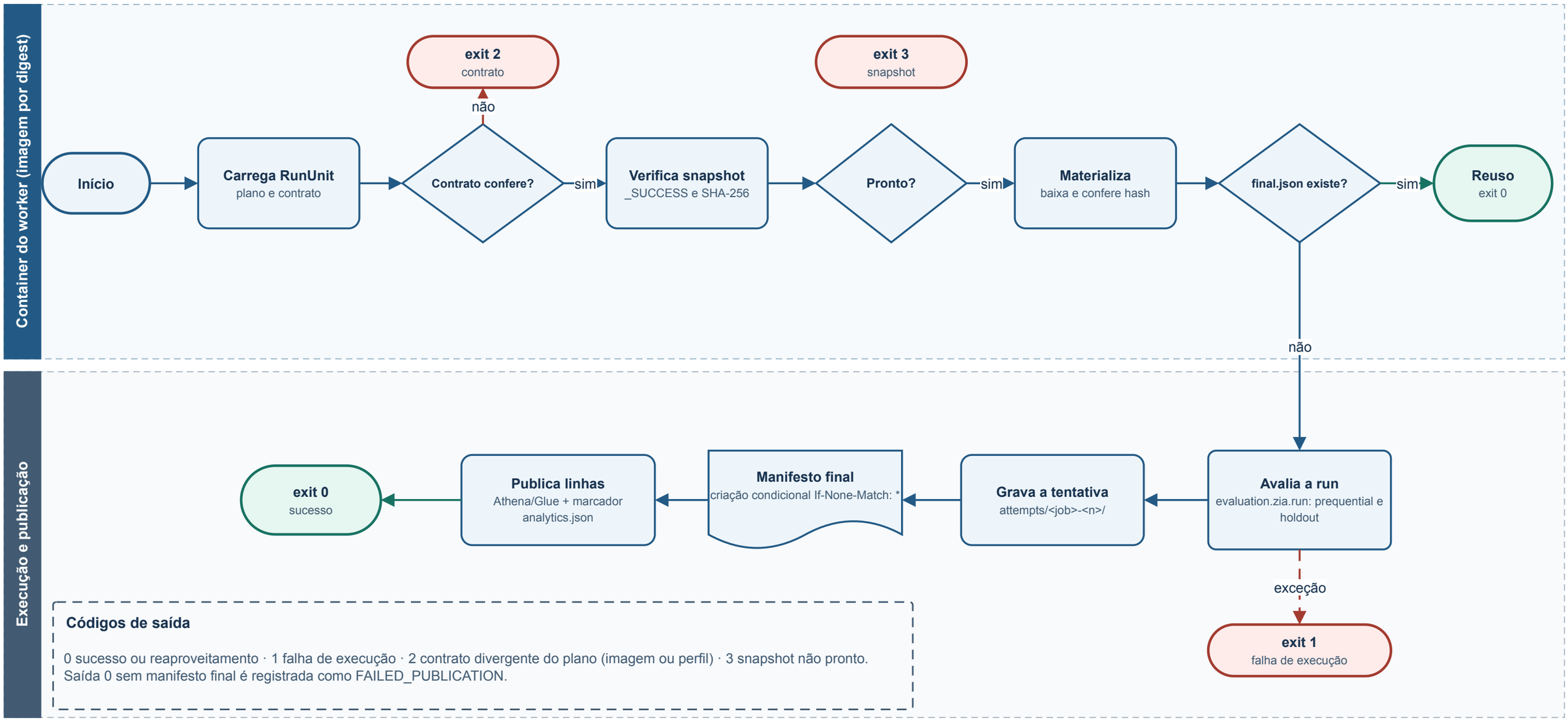
Jobs com nome determinístico cb-<campanha>-<run_id>. Jobs duplicados ativos são terminados e o mais antigo é mantido.

Uma saída 0 sem manifesto final conta como FAILED_PUBLICATION e a unidade volta à reconciliação até esgotar as tentativas.

O timeout da fila (RUNNABLE), AttemptDurationSeconds e TimeoutSecondsPath limitam espera e execução.

Uma run no job Batch: validar, avaliar, publicar

Cada job executa uma run e sai. O manifesto final é criado de forma condicional, então retentativas reaproveitam o que já foi concluído.



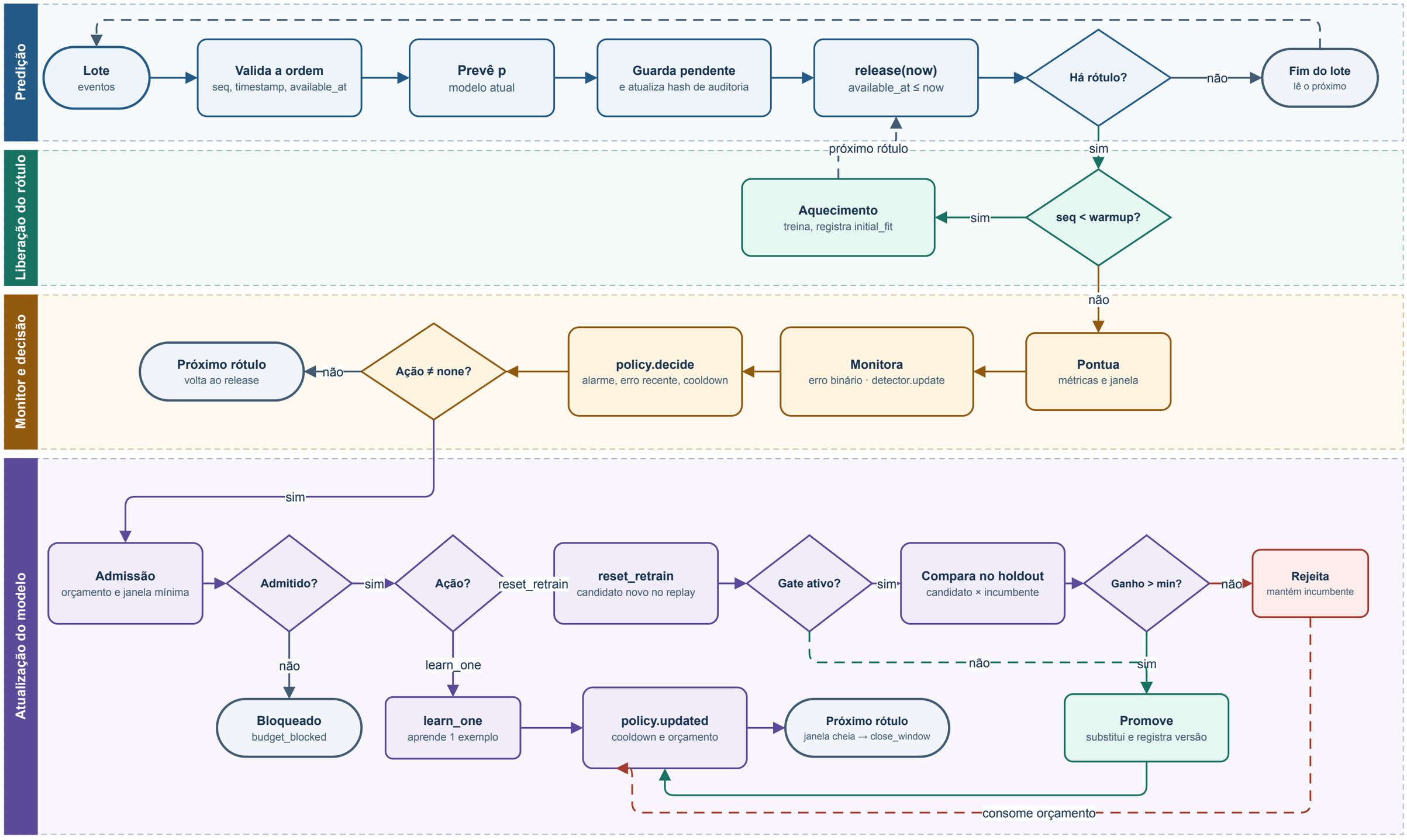
Cada tentativa grava a própria evidência em attempts/<job>-<n>/, incluindo as linhas do Athena. Tentativas falhas nunca apagam as anteriores.

O manifesto final usa If-None-Match: *. Depois dele, as linhas da tentativa vencedora são publicadas e o marcador analytics.json é gravado.

SIGTERM aciona o GracefulTerminationHandler. A reconciliação nunca termina jobs de outra campanha.

Laço prequential do engine

Todas as predições de um lote acontecem antes de qualquer aprendizado desse lote. O rótulo só é usado depois de liberado.



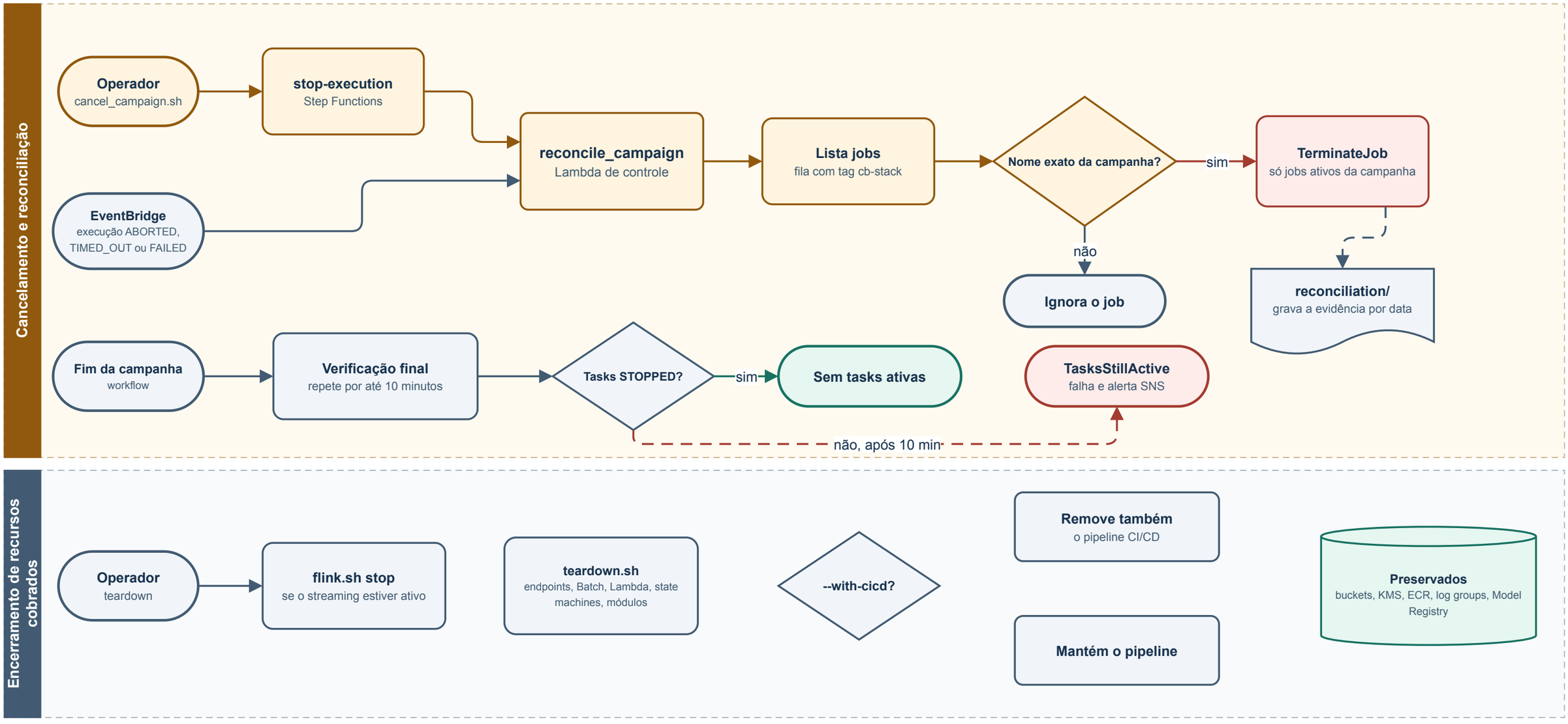
Sem promoção, todo treinamento concluído substitui o modelo. Com validation_samples, o candidato só é promovido se o ganho superar min_improvement.

Candidatos rejeitados também consomem orçamento e iniciam o cooldown. A falha de treino nunca substitui o incumbente.

O sinal do detector é o erro binário da predição feita antes do rótulo: é drift de desempenho, não prova mudança em P(Y|X).

Cancelamento, reconciliação e encerramento

Nenhuma campanha deixa tasks cobradas para trás: três gatilhos levam à mesma reconciliação, e o encerramento preserva os dados.



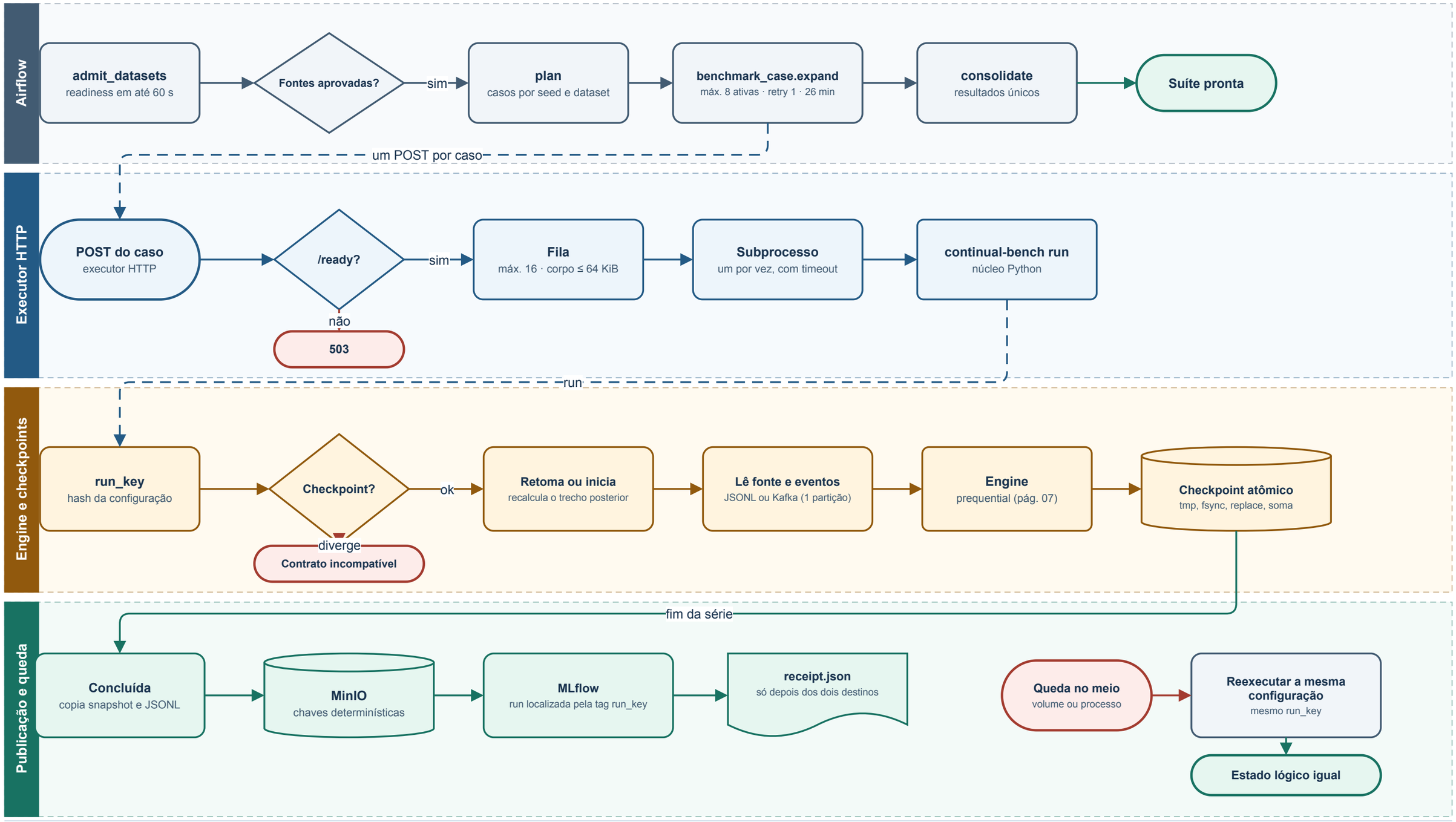
A interrupção do .sync é de melhor esforço. Por isso a regra do EventBridge chama reconcile_campaign em execuções ABORTED, TIMED_OUT ou FAILED.

TerminateJob só alcança jobs com a tag cb-stack e nome exato cb-<campanha>-<run_id de 20 hex>, para não atingir campanhas com prefixo igual.

Limites de tempo: jobStateTimeLimitActions cancela jobs presos em RUNNABLE; AttemptDurationSeconds limita a execução; TimeoutSecondsPath limita espera mais execução.

Perfil local: DAG, executor e recuperação

O Airflow só coordena tarefas finitas. O ciclo por amostra fica no núcleo Python, e a recuperação depende do run_key.



Uma queda entre MinIO e MLflow deixa um destino completo e o outro não. Repetir a publicação converge ao mesmo estado lógico, sem transação distribuída.

Kafka usa produtor idempotente com acks=all. Duplicatas são descartadas por sequência e lacunas, inversões ou conteúdo diferente da fonte falham.

Apenas uma réplica do executor é suportada. Pickle só aceita checkpoints criados por esta instalação confiável.